

Documentation technique — Vite & Gourmand

1. Réflexion technologique initiale

1.1 Choix du back-end : pourquoi PHP 8.2 sans framework ?

Le sujet ne contraint aucune technologie hormis l'usage d'une BDD relationnelle et d'une BDD NoSQL. Plusieurs options ont été évaluées :

Option	Avantages	Inconvénients
Laravel/Symfony	Productivité élevée, écosystème mature	Le jury veut vérifier que je maîtrise les fondamentaux ; framework trop "magique" pour un TP
Node.js + Express	Stack moderne	Hors de mon parcours Studi
PHP 8.2 + PDO + MVC maison 	Maîtrise totale du code, démontre la compréhension	Plus de code à écrire

Décision : PHP 8.2 + PDO + architecture MVC implémentée à la main, avec autoloading PSR-4 via Composer. Cela correspond exactement à la stack pédagogique de Studi.

1.2 Choix du front : pourquoi pas de framework JS ?

Le projet ne nécessite ni SPA ni interactions complexes côté client. **HTML / CSS / JS vanilla** suffit largement. Avantages :

- Aucune dépendance npm à gérer
- Page rendue côté serveur → SEO et accessibilité naturels
- Performances excellentes
- Démontre la maîtrise des fondamentaux (exigence du référentiel TP)

Pour les besoins JS spécifiques :

- **Filtres dynamiques sur la liste des menus** : `fetch()` + injection HTML
- **Calcul prix temps réel** sur le formulaire de commande : événements `input` + maths côté client
- **Graphique admin** : Chart.js via CDN (léger, ~80 ko)

1.3 Choix BDD relationnelle : MariaDB / MySQL

Imposé par l'énoncé. MariaDB 10.4 est livré avec XAMPP, donc utilisé en local. Il est interchangeable avec MySQL 8.x (même dialecte SQL). En production, peut être remplacé par PostgreSQL sans modification majeure (PDO abstrait le pilote).

1.4 Choix BDD NoSQL : MongoDB

Imposé par l'énoncé : « Les données [du graphique admin] doivent venir d'une base de données non relationnelle ».

Choisi **MongoDB** pour :

- Documentation abondante
- Driver PHP officiel
- Adapté aux **données semi-structurées** (statistiques avec historique embarqué)

Dans notre cas, MongoDB stocke :

- `commandes_par_menu` : un document par menu avec un compteur incrémenté + un tableau d'historique (date, montant)
- `logs` : audit trail (création commande, changement de statut, création employé, etc.)

C'est un usage **complémentaire** au SQL, pas concurrent.

2. Configuration de l'environnement

Pré-requis

Outil	Version	Rôle
PHP	8.2+	Exécution back-end
Extensions PHP	<code>pdo_mysql</code> , <code>mongodb</code> , <code>mbstring</code> , <code>openssl</code>	Indispensables
Composer	2.x	Gestion dépendances
MariaDB / MySQL	10.4+ / 8.0+	BDD relationnelle
MongoDB	6.x+	BDD NoSQL
Git	2.x+	Versioning

Installation locale

```
# 1. Cloner le dépôt
git clone https://github.com/EvanOROFINO/vitegourmand.git
cd vitegourmand

# 2. Installer les dépendances PHP
composer install

# 3. Configurer .env
cp .env.example .env
# (puis éditer si besoin)

# 4. Créer la BDD MySQL
mysql -u root < sql/01_schema.sql
mysql -u root < sql/02_seed.sql

# 5. Démarrer MongoDB (port par défaut 27017)
mongod --dbpath /chemin/vers/dossier/data

# 6. Lancer le serveur PHP
php -S localhost:8000 -t public
```

Stack VS Code recommandée

- Extension **PHP Intelephense**
- Extension **PHP DocBlocker**
- Extension **MySQL** (Weijan Chen)

3. Modèle conceptuel de données

Le MCD a été construit à partir de l'**Annexe 1 du sujet**, avec quelques ajouts pour respecter complètement les besoins métier.

Tables principales

```
utilisateur (utilisateur_id, email, password, prenom, nom, telephone,  
            adresse_postale, ville, pays, actif, reset_token, reset_expires)
```

```
role (role_id, libelle)
```

```
possede → relation N,N entre utilisateur et role
```

```

theme (theme_id, libelle)
regime (regime_id, libelle)

menu (menu_id, titre, description, nombre_personne_minimum, prix_par_personne,
      conditions_menu, quantite_restante, theme_id, regime_id, actif)

menu_image → galerie d'images d'un menu

plat (plat_id, titre, photo, type{entree|plat|dessert})

menu_plat → relation N,N entre menu et plat

allergene (allergene_id, libelle)
plat_allergene → relation N,N entre plat et allergene

commande (numero_commande PK varchar, utilisateur_id, menu_id,
          date_commande, date_prestation, heure_livraison,
          adresse_livraison, ville_livraison, distance_km,
          nombre_personne, prix_menu, prix_livraison, reduction, prix_total,
          statut, pret_materiel, restitution_materiel,
          motif_annulation, mode_contact)

commande_statut_historique (id, numero_commande, statut, commentaire, date_changer)

avis (avis_id, utilisateur_id, numero_commande, note, description, statut)

horaire (horaire_id, jour, heure_ouverture, heure_fermeture, ordre)

contact_message (id, titre, description, email, created_at)

```

Justification des écarts avec le MCD de l'annexe

- **Numéro de commande** = `VARCHAR(50)` au lieu d'un INT auto-incrémenté → permet un format `CMD-YYYYMMDD-XXXXXX` lisible par le client
- **Table commande_statut_historique** ajoutée pour stocker la timeline de la commande (exigence du sujet sur la traçabilité)
- **Table menu_image** ajoutée pour gérer la galerie (exigence : « une galerie d'image »)

Schéma MongoDB (NoSQL)

Collection `commandes_par_menu` :

```

{
  "_id": "ObjectId(...)",
  "menu_id": 1,

```

```

"titre": "Menu Tradition Bordelaise",
"nb_commandes": 5,
"ca_total": 1850.00,
"historique": [
  { "date": ISODate("2026-05-07"), "montant": 486.00 },
  ...
]
}

```

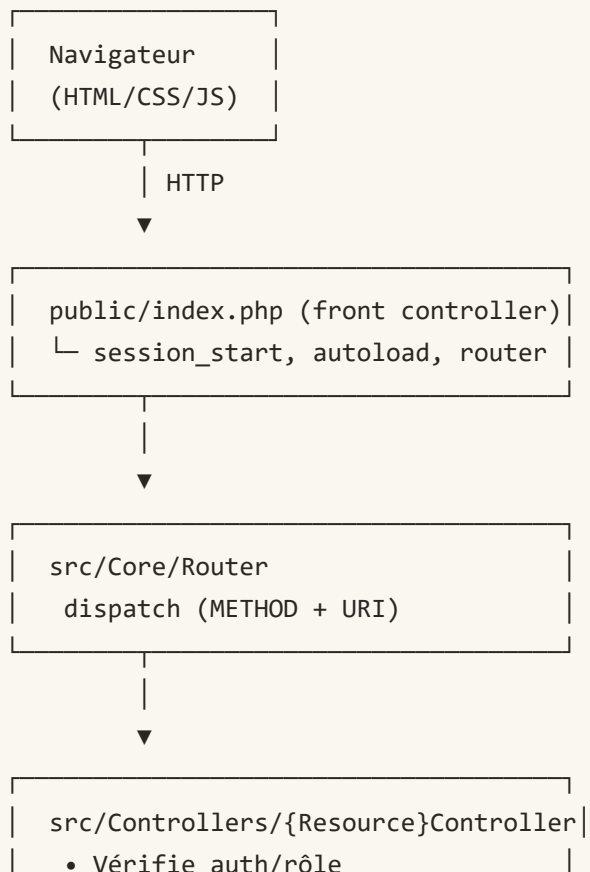
Collection `logs` :

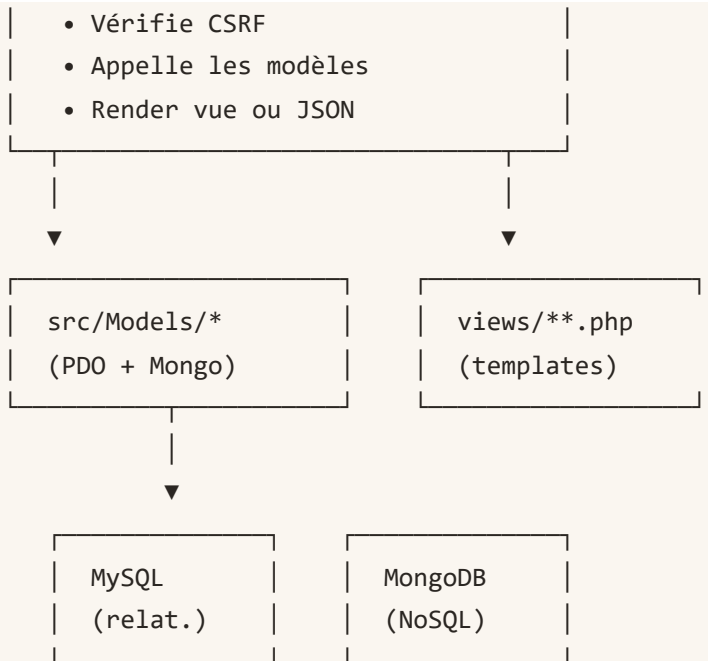
```

{
  "_id": "ObjectId(...)",
  "event": "commande_creee",
  "context": { "numero": "CMD-...", "user_id": 3 },
  "date": ISODate("..."),
  "ip": "127.0.0.1"
}

```

4. Architecture applicative





Flux de requête type — Création d'une commande

1. **Front** : utilisateur soumet le formulaire `POST /commander`
2. **Routeur** : match sur `[CommandeController::class, 'submit']`
3. **CommandeController** :
 - `Auth::requireLogin()` → redirige vers `/login` si non connecté
 - `verifyCsrf()` → 419 si jeton invalide
 - Validation des champs (date future, nombre min, ville, etc.)
 - `Commande::calculerPrix()` → applique règles métier (réduction, livraison)
 - `Commande::create()` → insert MySQL transactionnel (commande + historique)
 - `Stats::incrementCommandeMenu()` → upsert MongoDB
 - `Stats::log()` → audit trail MongoDB
 - `Mailer::send()` → mail de confirmation
 - `flash('success', ...)` + redirect

5. Sécurité

Risques OWASP top 10 et mitigations

Risque	Mitigation
A01 — Broken access control	Méthodes <code>Auth::requireRole()</code> sur chaque action protégée

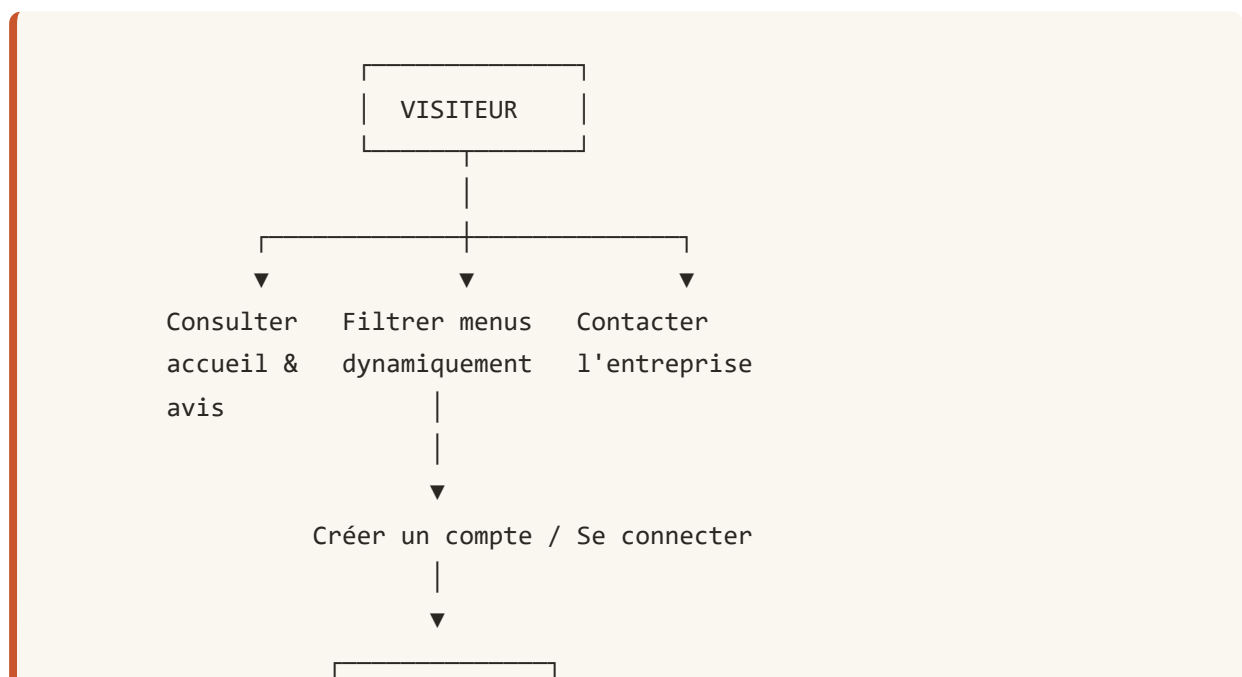
Risque	Mitigation
A02 — Cryptographic failures	<code>password_hash</code> avec bcrypt, pas de mots de passe en clair, <code>.env</code> exclu de git
A03 — Injection SQL	Requêtes préparées PDO avec paramètres nommés (zéro concat)
A03 — Injection XSS	Échappement HTML via <code>e()</code> partout, CSP en HTTP header
A04 — Insecure design	Mot de passe fort imposé (10 car., 4 classes), tokens uniques
A05 — Security misconfiguration	Erreurs masquées en prod (<code>APP_DEBUG=false</code>), <code>.htaccess</code> désactive <code>Indexes</code> , headers de sécurité
A07 — Identification & auth	<code>session_regenerate_id</code> au login, <code>samesite=Lax</code> , <code>httponly</code> , expiration tokens reset (1h)
A08 — Software integrity	<code>composer.lock</code> versionné, dépendances minimales
A10 — SSRF	Aucune fonctionnalité prenant des URL utilisateur

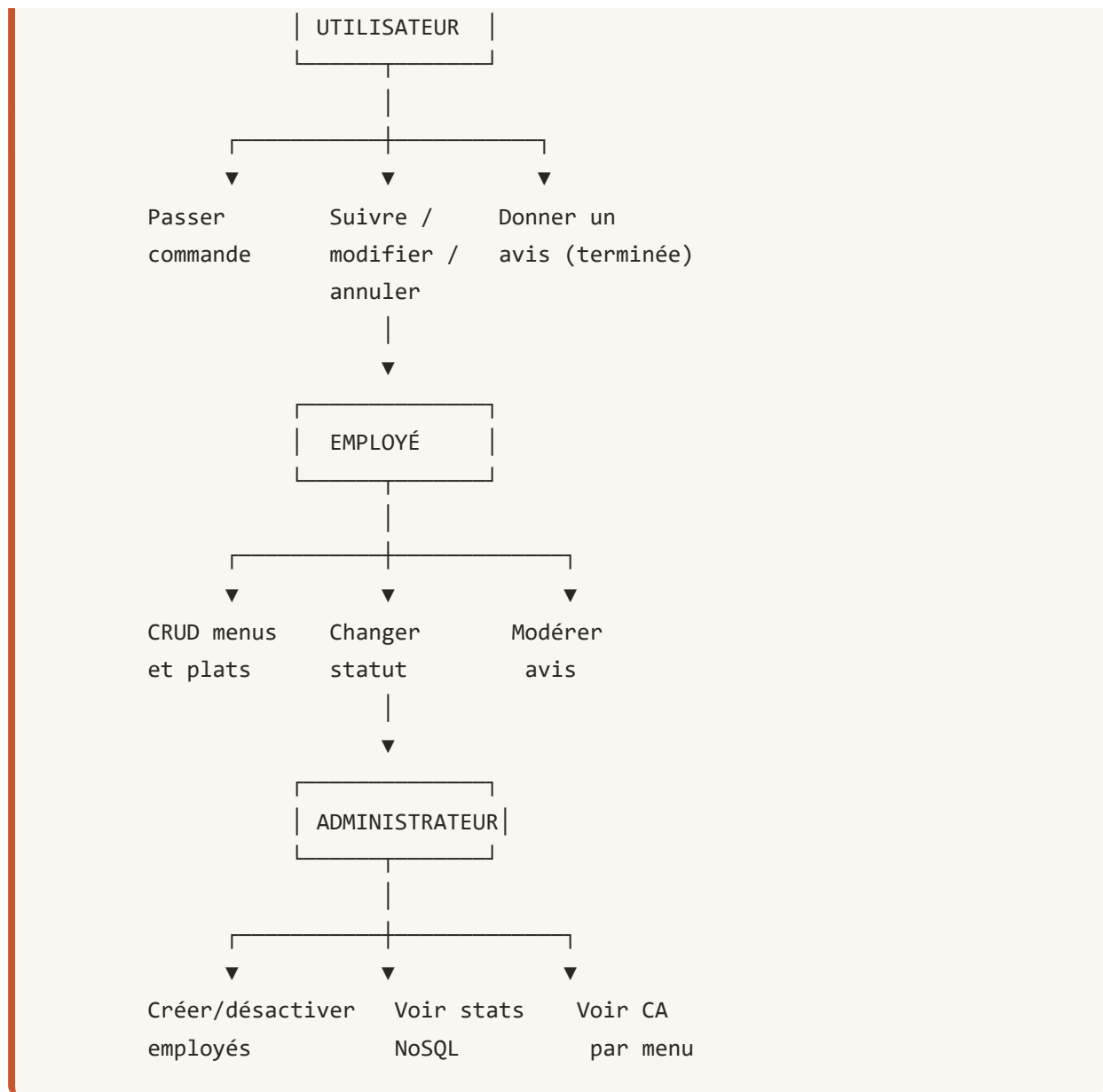
Secrets

Le fichier `.env` est dans `.gitignore` . La clé `APP_SECRET` est générée aléatoirement en production.

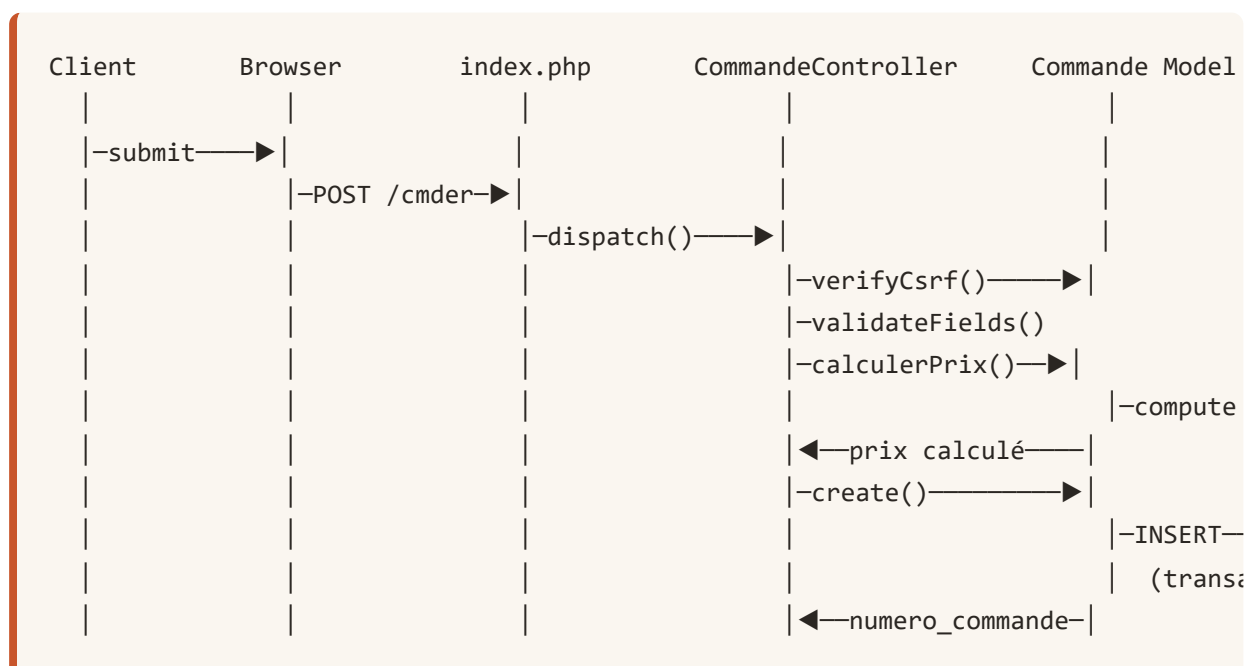
6. Diagrammes UML

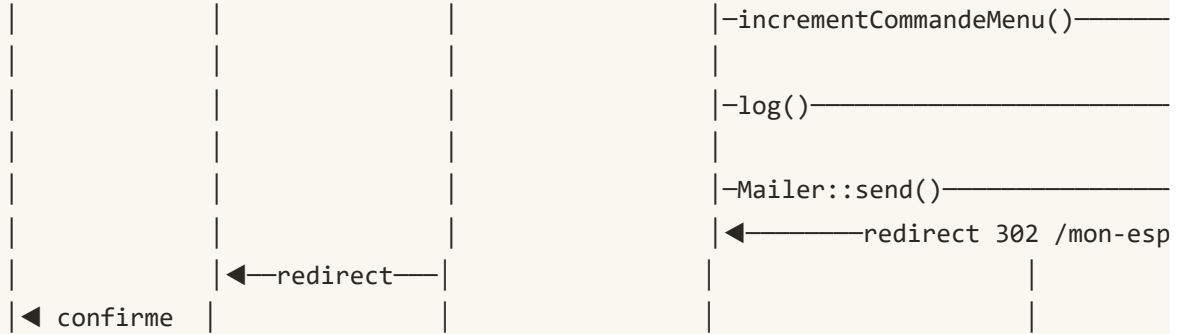
6.1 Diagramme de cas d'utilisation (Use Case)





6.2 Diagramme de séquence — Passage de commande





7. Documentation du déploiement

Plateformes cibles

Trois plateformes ont été envisagées :

Plateforme	PHP	MySQL	MongoDB	Coût
Railway	✓ via buildpack	✓ plugin	✓ via Atlas	gratuit (limité)
fly.io	✓ via Dockerfile	✓ via plugin	✓ via Atlas	gratuit (limité)
Render	✓ web service	✓ plugin	✓ via Atlas	gratuit (limité)

Choix retenu : (à compléter au moment du déploiement)

Variables d'environnement à définir en production

```
APP_ENV=production
APP_DEBUG=false
APP_URL=https://vitegurmand.example.com
APP_SECRET=<générer une clé aléatoire de 64 caractères>

DB_HOST=<host fourni par le provider>
DB_NAME=vitegurmand
DB_USER=<user>
DB_PASSWORD=<password>

MONGO_URI=mongodb+srv://<user>:<pass>@cluster.mongodb.net/
MONGO_DB=vitegurmand_nosql

MAIL_HOST=smtp.example.com
MAIL_PORT=587
```

```
MAIL_USERNAME=...
```

```
MAIL_PASSWORD=...
```

Étapes de déploiement (Railway exemple)

1. Créer un nouveau projet Railway connecté au repo GitHub `vitegourmand`
2. Ajouter un plugin **MySQL** → noter les variables `MYSQLHOST` , `MYSQLDATABASE` , etc.
3. Créer un compte **MongoDB Atlas** gratuit, créer un cluster M0, noter l'URI
4. Ajouter les variables d'env dans Railway
5. Configurer le **build command** : `composer install --no-dev --optimize-autoloader`
6. Configurer le **start command** : `php -S 0.0.0.0:$PORT -t public`
7. Push sur la branche `main` → déclenche le build
8. **Initialiser la BDD MySQL** : se connecter au shell Railway, exécuter `mysql < sql/01_schema.sql && mysql < sql/02_seed.sql`
9. Vérifier l'URL publique

Domaine personnalisé (optionnel)

Possibilité d'ajouter un domaine perso via les DNS du registrar (CNAME vers Railway).